



Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NACC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering

(NBA Accredited)

Subject- Applied Machine Learning for Industry Solutions (PECO311C)
Unit 1-FUNDAMENTALS OF APPLIED MACHINE LEARNING

Dr. A. V. Brahmane
Assistant Professor





Contents

Introduction to Machine Learning Overview of machine learning concepts, types (supervised, unsupervised, reinforcement), and real-world applications in industry.

Mathematical Foundations Basics of linear algebra (vectors, matrices), calculus (derivatives, gradients), and probability theory essential for understanding ML algorithms.

Data Preprocessing and Exploration Techniques for data cleaning, normalization, handling missing values, and exploratory data analysis.

Introduction to Supervised Learning Algorithms Linear Regression, Logistic Regression, Decision Trees, and Support Vector Machines, including their mathematical underpinnings and applications. Model Evaluation and Validation Techniques Understanding overfitting, underfitting, cross-validation methods.



Why “Learn” ?

- Machine learning is programming computers to optimize a performance criterion using example data or past experience.
- For **Enrich The System Behaviour**
- There is no need to “learn” to calculate payroll
- Learning is used when:
 - Human expertise does not exist (navigating on Mars),
 - Humans are unable to explain their expertise (speech recognition)
 - Solution changes in time (routing on a computer network)
 - Solution needs to be adapted to particular cases (user biometrics)



What is Machine Learning?

A Computer Programme(Machine) is said to learn from experience E with respect to some class of Task T and performance measure P,if it perormance at tasks T,as measured by P,improves with experience E.

Learning from data: a method of teaching machines to improve predictions from data

. Can be thought of as a method for computers to recognize patterns

A natural outgrowth of the intersection of computer science and statistics

Optimize a performance criterion using example data or past experience.

A computer Program which learns from experience is called a **Machine Learning Program** or simply Learning Program.Such a program some time reffered as **Learner**.



Types of Machine Learning

1. Supervised Learning:

- **Definition:** The algorithm is trained on a labeled dataset, where each input is paired with a corresponding output (e.g., predicting house prices based on size and location).
- **Key Tasks:**
 - **Classification:** Predicting a discrete label (e.g., spam vs. not spam).
 - **Regression:** Predicting a continuous value (e.g., temperature).
- **Examples:** Linear regression, logistic regression, support vector machines (SVM), neural networks.
- **Applications:** Image classification, spam detection, medical diagnosis.



Supervised Learning

- Supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm to learn the mapping function from the input to the output.

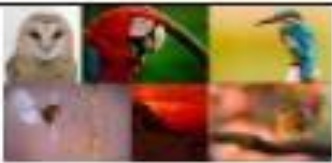
$$Y = f(X)$$

- The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.



- It is called supervised learning because the process of an algorithm learning from the training dataset can be thought of as a teacher supervising the learning process.
- We know the correct answers, the algorithm iteratively makes predictions on the training data and is corrected by the teacher.
- Learning stops when the algorithm achieves an acceptable level of performance.

Supervised Learning

Input	Label
	Cat
	Dog
	Fish
	Bird
	?



- Supervised learning problems can be further grouped into regression and classification problems.
- **Classification:** A classification problem is when the output variable is a **category**, such as “red” or “blue” or “disease” and “no disease”.
- **Regression:** A regression problem is when the output variable is a **real value**, such as “dollars” or “weight”.
- Some common types of problems built on top of classification and regression include recommendation and time series prediction respectively.
- Some popular examples of supervised machine learning algorithms are:
- **Linear regression** for regression problems.
- **Random forest** for classification and regression problems.
- **Support vector machines** for classification problems.



2. Unsupervised Learning:

- **Definition:** The algorithm works with unlabeled data to find hidden patterns or structures (e.g., grouping customers based on purchasing behavior).
- **Key Tasks:**
 - **Clustering:** Grouping similar data points (e.g., k-means clustering).
 - **Dimensionality Reduction:** Reducing the number of features while preserving information (e.g., principal component analysis, PCA).
- **Examples:** K-means clustering, hierarchical clustering, autoencoders.
- **Applications:** Customer segmentation, anomaly detection, data compression.



Unsupervised Machine Learning

- Unsupervised learning is where you only have input data (X) and **no corresponding output variables**.
- The goal for unsupervised learning is to model the underlying structure or distribution in the data in order **to learn more about the data**.
- These are called unsupervised learning because unlike supervised learning above there **is no correct answers** and there **is no teacher**. Algorithms are left to their own devices to discover and present the interesting structure in the data.

Unsupervised Learning

Input	Clusters
	



- Unsupervised learning problems can be further grouped into clustering and association problems.
- **Clustering:** A clustering problem is where you want to discover the inherent **groupings** in the data, such as grouping customers by purchasing behavior.
- **Association:** An association rule learning problem is where you want to **discover rules** that describe large portions of your data, such as **people that buy X also tend to buy Y.**
- Some popular examples of unsupervised learning algorithms are:
- k-means for clustering problems.
- Apriori algorithm for association rule learning problems.



Learning Associations

□ Basket analysis:

$P(Y | X)$ probability that somebody who buys X also buys Y where X and Y are products/services. Example: $P(\text{chips} | \text{beer}) = 0.7$

Transaction	Items in basket
1	milk, bananas, chocolate
2	milk, chocolate
3	milk, bananas
4	chocolate
5	chocolate
6	milk, chocolate

SOLUTION:

milk $\rightarrow$ bananas : Support = 2/6, Confidence = 2/4

bananas $\rightarrow$ milk : Support = 2/6, Confidence = 2/2

milk $\rightarrow$ chocolate : Support = 3/6, Confidence = 3/4

chocolate $\rightarrow$ milk : Support = 3/6, Confidence = 3/5

3. Reinforcement Learning:

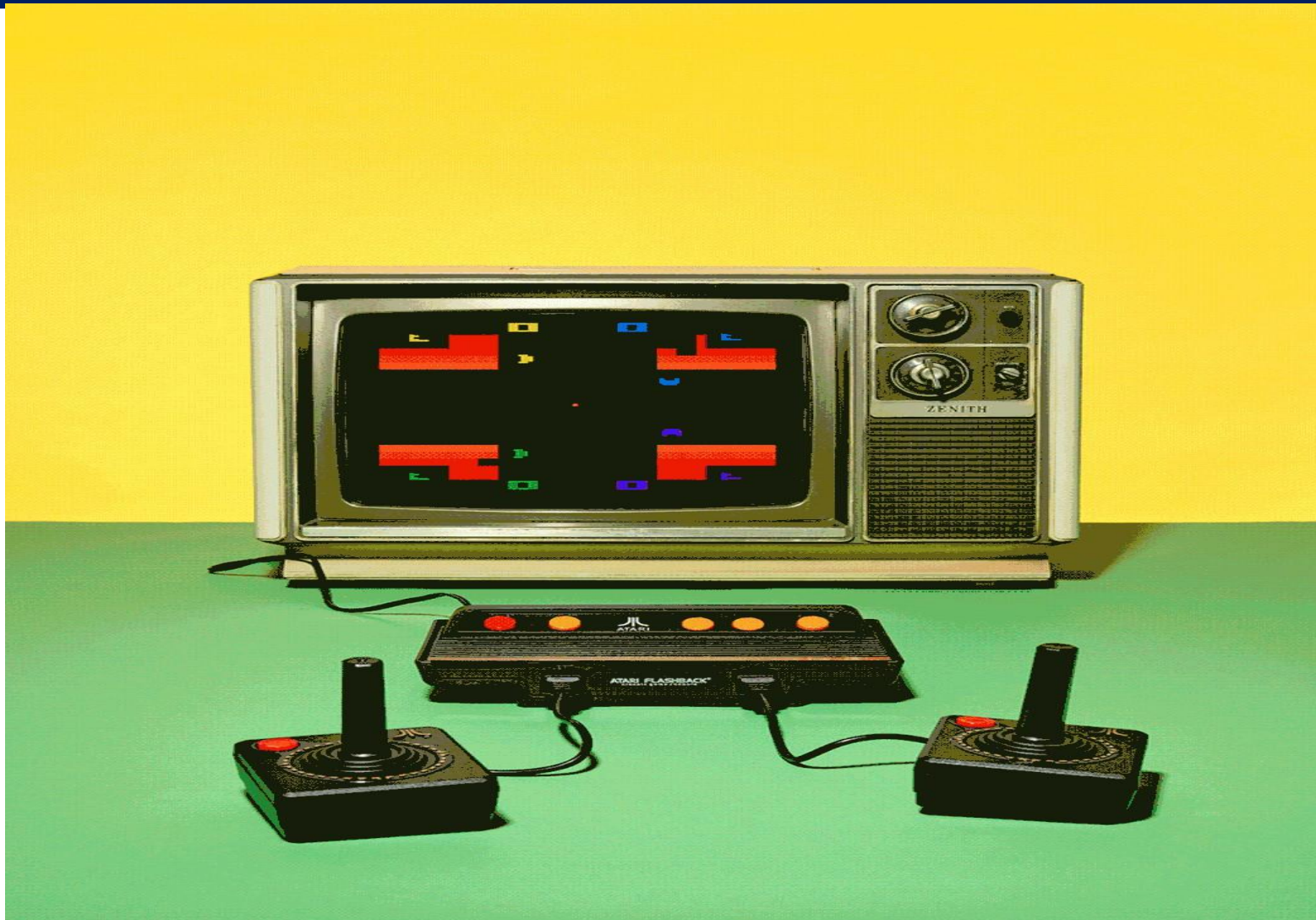
- **Definition:** The algorithm learns by interacting with an environment, receiving rewards or penalties based on actions (e.g., a robot learning to navigate a maze).
- **Key Components:**
 - **Agent:** The learner or decision-maker.
 - **Environment:** The system the agent interacts with.
 - **Reward:** Feedback signal to guide learning.
- **Examples:** Q-learning, deep reinforcement learning (e.g., AlphaGo).
- **Applications:** Game playing, robotics, autonomous vehicles.



Reinforcement learning

- Reinforcement learning is also based on feedback provided by the environment. However, in this case, the information is more qualitative and doesn't help the agent in determining a precise measure of its error.
- this feedback is usually called **reward** (sometimes, a negative one is defined as a penalty) and it's useful to understand whether a certain action performed in a state is positive or not.
- Reinforcement Learning is a framework for learning where an agent interacts with an environment and receives a reward for each interaction. The goal is to learn to accumulate as much reward as possible over time.

Reinforcement learning ex: Atari Video Game





Summary

Type	Description	Example Applications
Supervised Learning	Learns from labeled data (input-output pairs) to predict future outcomes.	Email spam detection, image classification, credit scoring
Unsupervised Learning	Finds hidden patterns or structures in unlabeled data.	Customer segmentation, anomaly detection, clustering
Reinforcement Learning	Learns optimal actions by interacting with an environment and receiving rewards or penalties.	Robotics, game AI, recommendation systems



Application of Machine Learning

In **retail business**, machine learning is used to study consumer behaviour.

In **finance**, banks analyze their past data to build models to use in credit applications, fraud detection, and the stock market.

In **manufacturing**, learning models are used for optimization, control, and troubleshooting.

In **medicine**, learning programs are used for medical diagnosis.

In **telecommunications**, call patterns are analyzed for network optimization and maximizing the quality of service.

In **science**, large amounts of data in physics, astronomy, and biology can only be analyzed fast enough by computers. The World Wide Web is huge; it is constantly growing and searching for relevant information cannot be done manually.



Applications Continuation

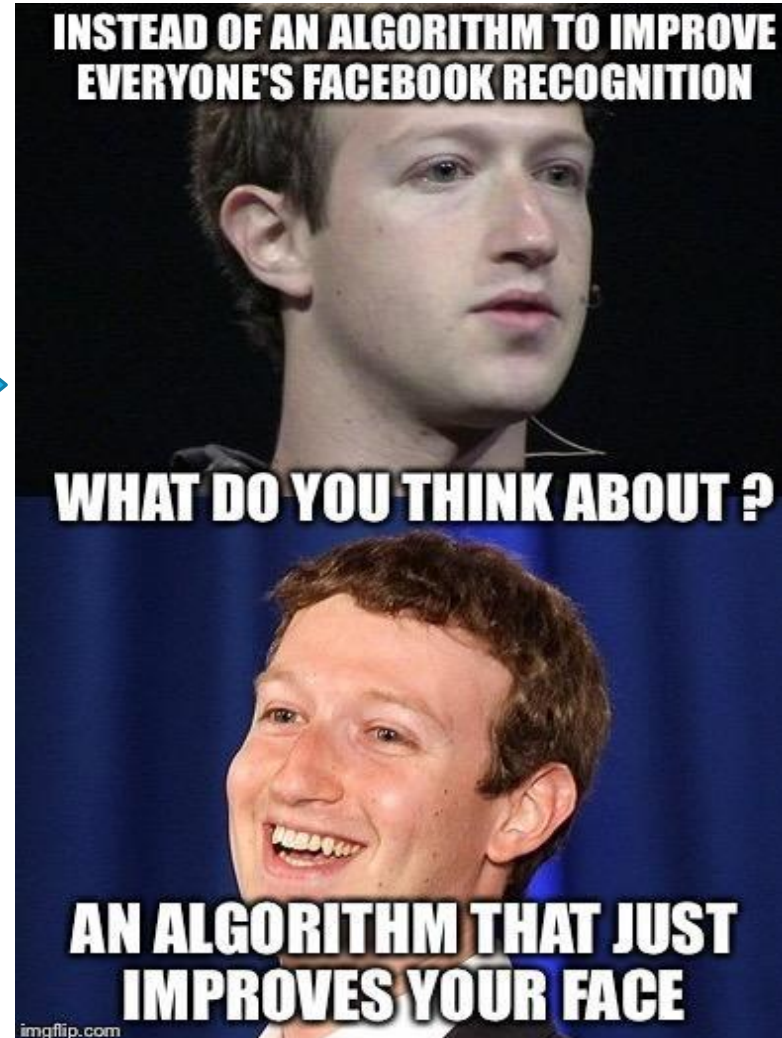
7. In **artificial intelligence**, it is used to teach a system to learn and adapt to changes so that the system designer need not foresee and provide solutions for all possible situations.
8. It is used to find solutions to many problems in **vision, speech recognition, and robotics**.
9. Machine learning methods are applied in the **design of computer-controlled vehicles** to steer correctly when driving on a variety of roads.
10. Machine learning methods have been used to develop programmes for **playing games** such as chess, backgammon and Go.

Face Recognition

Face Recognition measures similarity between human faces and recognizes a person's identity from frontal facial images by comparing it to faces previously stored in a



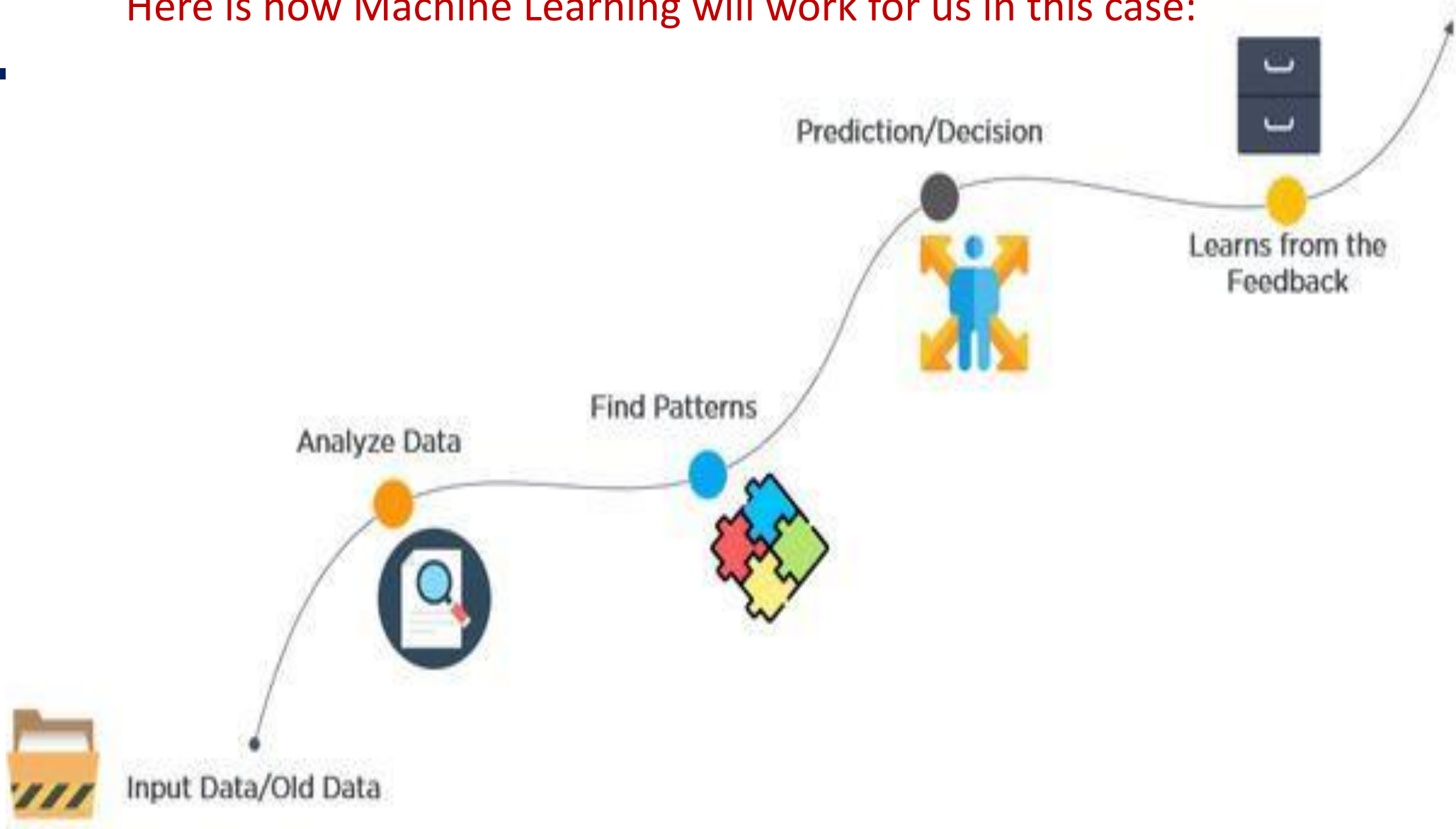
Or
even
do
this.....





- Let us say we have a **dataset** that contains pictures of different kinds of **fruits** and we want Machine Learning to **segregate the photos based on the kind of fruits**.
- First we provide the dataset to the system i.e we provide the input data.
- The system goes through the entire dataset or analyses **it to find patterns based on size, shapes, colors, etc.**
- Now that it has figured out the patterns, the **systems takes decisions and starts separating the photos based on the patterns.**
- Once the work is done, the **system learns from the feedback it gets.** If it gets any of the fruit type wrong, it will make sure it does not happen in the future.

Here is how Machine Learning will work for us in this case:





Key Components of ML Systems

Component	Description
Data	Input collected from real-world phenomena (text, images, numbers, etc.)
Features	Measurable properties or characteristics of the data
Model	The mathematical structure or algorithm used to make predictions
Training	Process of teaching the model using historical data
Prediction	Output generated by the trained model on new data
Evaluation	Checking how well the model performs (e.g., accuracy, precision)



Mathematical Foundations

A solid understanding of the following mathematical concepts is essential for ML:

•Linear Algebra

- **Vectors:** Ordered arrays of numbers representing features or data points.
- **Matrices:** 2D arrays used for representing datasets and transformations.

•Calculus

- **Derivatives:** Measure how a function changes as its input changes; used in optimization (e.g., gradient descent).
- **Gradients:** Vector of partial derivatives; points in the direction of steepest increase of a function.

•Probability Theory

- **Random Variables:** Variables whose values depend on outcomes of random phenomena.
- **Probability Distributions:** Describe how probabilities are distributed over values.
- **Conditional Probability:** Probability of an event given another event has occurred.

These foundations are crucial for understanding how ML algorithms work, especially in model training and evaluation.



Vectors

Concept: A vector is an array of numbers in 1D space. Think of it as a point or direction in space.

👉 Operations: Addition, Scalar Multiplication, Dot Product Norm (Length)

```
import numpy as np

# Define two vectors
v1 = np.array([2, 3])
v2 = np.array([1, 4])

# Vector addition
print("Addition:", v1 + v2)

# Dot product
print("Dot product:", np.dot(v1, v2))

# Norm (magnitude)
print("Norm of v1:", np.linalg.norm(v1))
#It calculates the Euclidean norm, using the formula
print("Norm of v2:", np.linalg.norm(v2))
```

```
Addition: [3 7]
Dot product: 14
Norm of v1: 3.605551275463989
Norm of v2: 4.123105625617661
```



Matrices Concept: A matrix is a 2D array. It's used to represent systems of equations, linear transformations, and datasets.

👉 Operations: Transpose Matrix Multiplication Inverse (if square & non-singular)

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("Matrix Multiplication:\n", np.dot(A, B))
print("Transpose of A:\n", A.T)
print("Inverse of A:\n", np.linalg.inv(A))
```

Matrix Multiplication:

```
[[19 22]
```

```
[43 50]]
```

Transpose of A:

```
[[1 3]
```

```
[2 4]]
```

Inverse of A:

```
[[-2.  1. ]
```

```
[ 1.5 -0.5]]
```



What are Eigenvalues?

Given a square matrix A , an eigenvalue λ is a number such that there exists a non-zero vector v (called an eigenvector) satisfying: $Av = \lambda v$

This means when you multiply

A by v , the vector just gets scaled by λ , not rotated or changed direction.

How to calculate eigenvalues with NumPy?

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues(The values of  $\lambda$ ):", eigenvalues)
print("Eigenvectors(Corresponding eigenvectors  $v$  as columns):\n", eigenvectors)
```

```
Eigenvalues(The values of  $\lambda$ ): [-0.37228132  5.37228132]
Eigenvectors(Corresponding eigenvectors  $v$  as columns):
[[-0.82456484 -0.41597356]
 [ 0.56576746 -0.90937671]]
```

Interpretation:

- If you multiply A by the first eigenvector, the result is the same eigenvector scaled by approximately -0.3723 :

$$A\mathbf{v}_1 = -0.3723 \times \mathbf{v}_1$$

- Similarly for the second eigenvector and eigenvalue:

$$A\mathbf{v}_2 = 5.3723 \times \mathbf{v}_2$$



CALCULUS – Understanding Optimization Concept: Measures how a function changes — core to gradient descent.

```
import sympy as sp
```

```
x = sp.Symbol('x')  
f = x**2 + 3*x + 5
```

```
# Derivative
```

```
f_prime = sp.diff(f, x)  
print("f'(x):", f_prime)
```

```
f'(x): 2*x + 3
```

Gradients (Multivariate Derivatives) Concept: A vector of partial derivatives. Used for optimizing multivariable functions. To minimize loss functions (i.e., how wrong a model is). How gradients help: The gradient tells you the direction of steepest increase of the function.

To minimize a function (e.g., loss), you move in the opposite direction of the gradient.

```
x, y = sp.symbols('x y')  
f = x**2 + 3*y**2 + 2*x*y
```

```
# Gradient
```

```
grad = [sp.diff(f, var) for var in (x, y)]  
print("Gradient:", grad)
```

```
Gradient: [2*x + 2*y, 2*x + 6*y]
```

This creates a list with partial derivatives:

Partial derivative w.r.t. x: $=2x+2y$ Partial derivative w.r.t.y: $=6y+2x$ So, the gradient vector is:

$\nabla f(x,y)=[2x+2y, 6y+2x]$

Chain Rule & Partial Derivatives Concept: Used when functions are composed, especially in neural networks. These derivatives are used in gradient-based methods like backpropagation in neural networks, where such expressions arise often.

📌 Purpose: To compute gradients of the loss function with respect to all weights in a deep neural network.

💡 Role of gradients: Backpropagation uses the chain rule and partial derivatives to efficiently compute gradients layer by layer. These gradients are then used in gradient descent to update the weights.

```
u, v = sp.symbols('u v')
f = sp.sin(u * v)

df_du = sp.diff(f, u)
df_dv = sp.diff(f, v)

print("∂f/∂u:", df_du)
print("∂f/∂v:", df_dv)
```

```
∂f/∂u: v*cos(u*v)
∂f/∂v: u*cos(u*v)
```



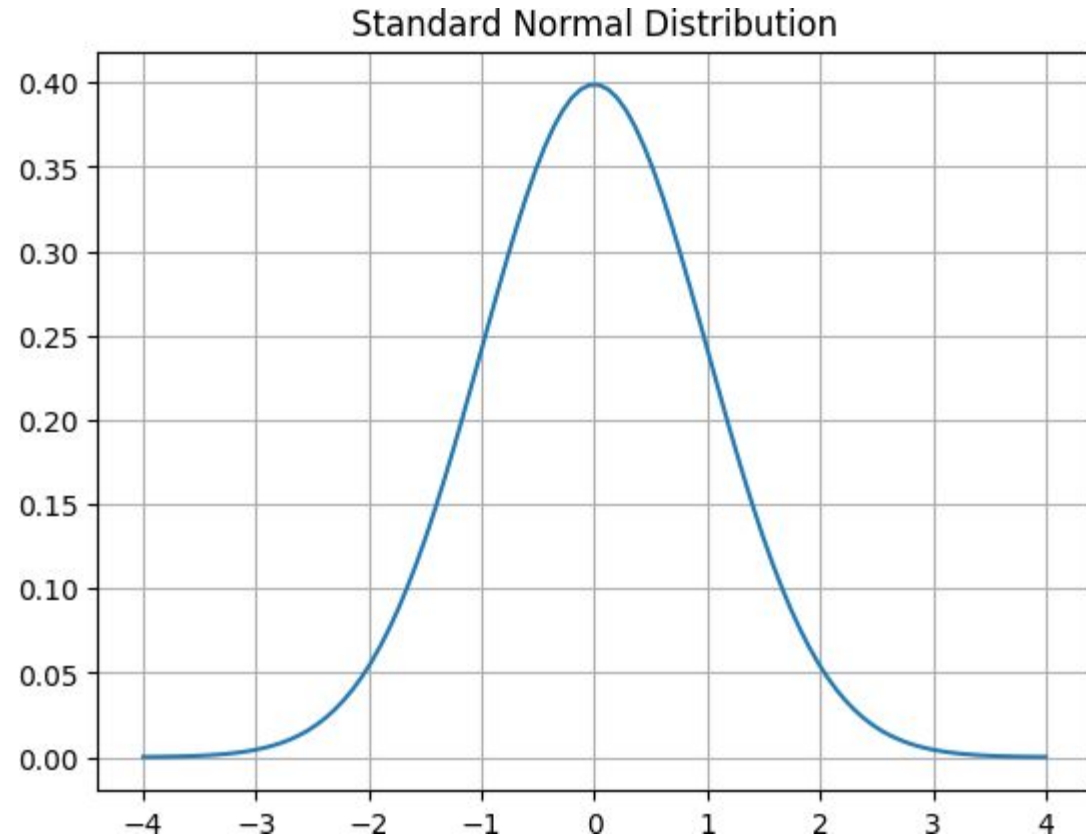
PROBABILITY THEORY For Uncertainty in ML

Probability Distributions Normal Distribution: Most common. Central to linear regression & many algorithms.

```
from scipy.stats import norm
import matplotlib.pyplot as plt
import numpy as np

x_vals = np.linspace(-4, 4, 100)
pdf = norm.pdf(x_vals, loc=0, scale=1)

plt.plot(x_vals, pdf)
plt.title("Standard Normal Distribution")
plt.grid(True)
plt.show()
```





Expectation & Variance Concept: Expectation (mean): Average value

Variance: $\sigma^2 = \sum (x_i - \mu)^2 / N$

```
data = np.array([1, 2, 3, 4, 5])
print("Mean:", np.mean(data))
print("Variance:", np.var(data))
```

Mean: 3.0

Variance: 2.0

Bayes' Theorem Used for: Naive Bayes, probabilistic reasoning, etc.

```
# P(A|B) = P(B|A) * P(A) / P(B)

P_A = 0.01          # Disease prevalence
P_B_given_A = 0.95  # Test positive if diseased
P_B_given_not_A = 0.1
P_not_A = 1 - P_A

# Total probability of positive test
P_B = P_B_given_A * P_A + P_B_given_not_A * P_not_A

# Bayes Theorem
P_A_given_B = (P_B_given_A * P_A) / P_B
print(f"P(Disease | Positive Test): {P_A_given_B:.4f}")
```

P(Disease | Positive Test): 0.0876



Data Preprocessing Overview

Data preprocessing is a crucial step in the machine learning pipeline. It ensures the data is clean, consistent, and suitable for training models.

Goals:

- Improve data quality
- Reduce noise and redundancy
- Handle inconsistencies
- Prepare data for modeling

What is Data Cleaning?

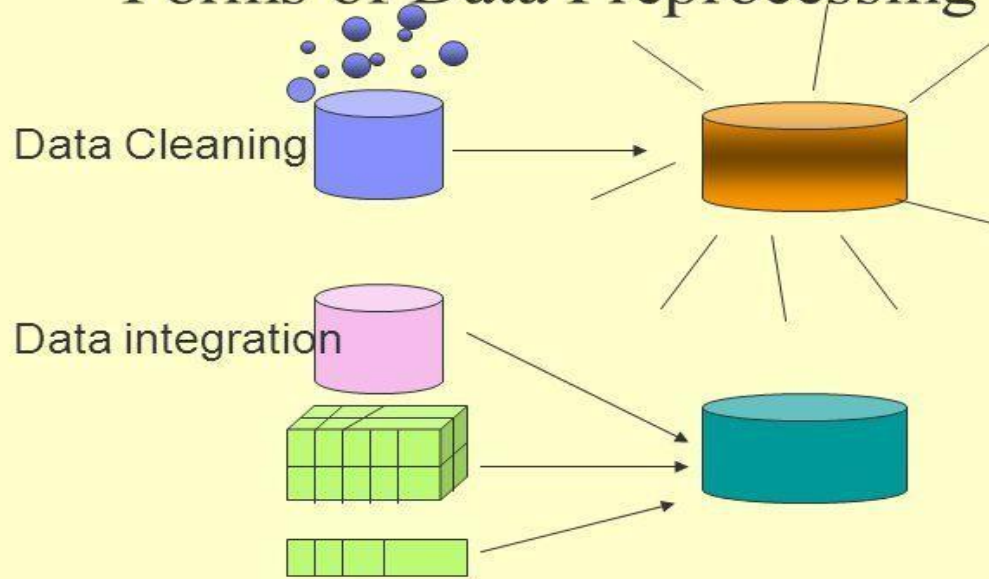
Data cleaning involves fixing or removing incorrect, corrupted, or incomplete records.

Common Issues in Raw Data:

- Missing values
- Duplicates
- Inconsistent formatting (e.g., "yes", "Yes", "Y")
- Outliers

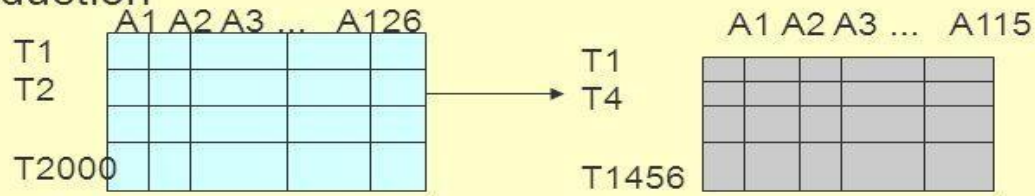
How to Handle Missing Data?

Forms of Data Preprocessing



Data transformation -2, 32, 100, 59, 48 → -0.02, 0.32, 1.00, 0.59, 0.48

Data reduction



- Ignore the tuple with missing data values :
- Manually filling the missing data values:
- Using global constant values
- Using attribute average values
- Using most likely desirable values



Noisy Data

- Noise: random error or variance in a measured variable
- Incorrect attribute values may due to
 - faulty data collection instruments
 - data entry problems
 - data transmission problems
 - technology limitation
 - inconsistency in naming convention
- Other data problems which requires data cleaning
 - duplicate records
 - incomplete data
 - inconsistent data



How to Handle Noisy Data?

- Binning method:
 - first sort data and partition into (equi-depth) bins
 - then smooth by bin means, smooth by bin median, smooth by bin boundaries, etc.
- Clustering
 - detect and remove outliers
- Combined computer and human inspection
 - detect suspicious values and check by human
- Regression
 - smooth by fitting the data into regression functions



Smoothing Noisy Data Using Binning Method:

- This method removes the noisy data value from the sorted data set by communicating with its neighboring values.
- It is considered local smoothing because it communicates with the values which are around it.
- Before initiating any of the binning method, it is necessary to sort the given list either in increasing order or decreasing order.
- This sorted list is then divided into a number of buckets with equivalent frequency or width.



Binning process can be accomplished by using the following approaches.

1. Smoothing by calculating the average mean of a bucket.
2. Smoothing by calculating the median of a bucket.
3. Smoothing by considering the boundaries of buckets.



Smoothing by calculating the average mean of a bucket.

This method calculates the average of data values for individual bucket and substitutes the existing values with the average values.

For ex: the following sorted list which is divided into buckets of equivalent frequency.

2,3,5,6,8,9,9,10,12,15,17,20.

Bucket A : 2,3,5,6

B : 8,9,9,10

C: 12,15,17,20.

The averages of buckets A,B,C are 4,9,16 respectively.

The existing values in the buckets are substituted as

Bucket A : 4,4,4,4

B : 9,9,9,9

C: 16,16,16,16



Smoothing by calculating the Median of bucket.

This method similar to that of bucket mean method except that median is calculated for individual buckets.

For ex: the following sorted list which is divided into buckets of equivalent frequency.

2,3,5,6,8,9,9,10,12,15,17,20.

Bucket A : 2,3,5,6

B : 8,9,9,10

C: 12,15,17,20.

The median of the above list for buckets A,B,C are 4,9,16 respectively which are substituted in the following way.

Bucket A : 4,4,4,4

B : 9,9,9,9

C: 16,16,16,16



Smoothing by considering the Boundaries of bucket.

In this method, *the highest and lowest values* in individual buckets are considered.

These values are treated as *boundaries of a bucket*.

Every data value in the bucket is substituted by the boundary value which is nearest to it. The highest and lowest values remain unchanged.

The highest value of buckets A,B,C are 6,10,20 and the lowest values are 2,8,12 respectively.

The values in the buckets are substituted in the following manner.

Bucket A : 2,2,6,6

B : 8,8,8,10

C: 12,12,20,20



Smoothing Noisy Data Using Regression Method

Regression is function which is used to remove noisy data. There are two types of regression functions which are used for this purpose.

1. **Linear regression function.**
2. **Multiple linear regression function.**

This function is responsible for searching the line that is most suitable to model the correlation between two attribute values.

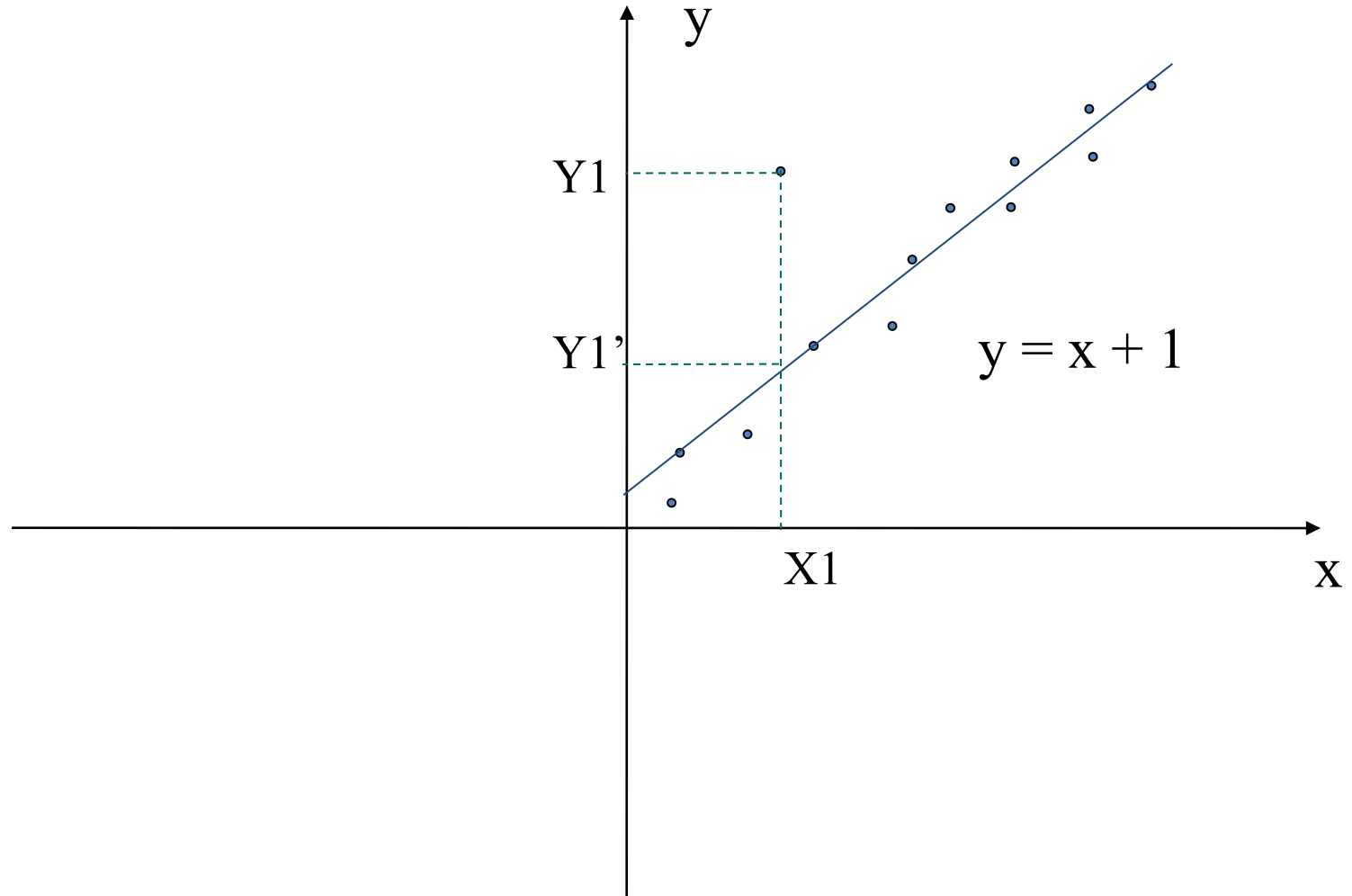
This relation is made in such a way that these attributes are dependent on each other. i.e one attribute can be utilized to estimate the value of other attributes

Multiple linear regression function.

This function is an enhancement of simple linear regression function. In this, at least two attribute values are considered and the data associated with this function is pertinent to a multidimensional surface.



Regression





Smoothing Noisy Data Using Clustering Approach

Clustering is a method of combining abstract object sets into classes of identical objects.

A cluster is defined as a group of data objects that have high similarity if they belong to same cluster and are dissimilar to objects belonging to other clusters.

Clustering can be *used to delete the outliers* which are the data values that usually exhibit dissimilar nature in a particular cluster.



Normalization of data

Normalization is a process of decomposing the attribute values so that they are within a specified range of small size. Normalization is used in,

- Neural network classification algorithm such as back-propagation algorithm that enhances the speed of learning phase.
- Distance-based method such as k-nearest neighbour classification that prohibit the larger range attribute values from out weighing smaller range attribute values.



The different data normalization techniques are,

i.Min-Max Normalization:

This type of data normalization technique is responsible for accomplishing linear transformation on actual data set and for retaining the correlation between them. Let the minimum and maximum values of attribute P be represented as M_p and X_p respectively. In min-max normalization, a value d of attribute P is mapped to value D in the range $[nX_p, nM_p]$ by calculating D using formula,

$$D = \{[d - M_p] / [X_p - M_p]\} (nX_p - nM_p) + nM_p$$

An error “out-of-bound” is displayed if the input value is greater than the actual data range.

Ex: Consider that min and max value attribute price are 3000 and 5000 respectively and the range is [0.5 -2.0]. Then the value 7000 is transformed using min-max normalization as,

$$D = [7000-3000]/[5000-3000](2.0-0.5)+0.5=4000/2000(1.5)+0.5=3.5$$

Out-of- bound error is encountered, as the value is greater than actual interval range.



ii. Zero-Score (z-score) Normalization:

This method is generally used when the actual min and max data values are not known or When these values are considered as outliers. In this method, arithmetic mean and standard deviation are used for performing normalization on attribute P. the value d is transformed to D using,

$$D = [d - P_{AM}] / P_{SD}$$

where P_{AM} Arithmetic mean of attribute P.

P_{SD} Standard deviation of attribute P.

EX: Let us consider arithmetic mean of P is 32 standard deviation is 24. Then the value is 70 is transformed into

$$(70-32)/24=1.58$$



iii. Normalization Using Decimal Scaling:

The data value of attribute P is normalized by changing the position of decimal points.

The decision regarding where the decimal point should be placed is based on maximum

Absolute value of attribute i.e $\text{Max}(|D|)$.

The value d is transformed D using,

$$D = [d / 10^z]$$

where z represents the smallest integer value such that $\text{Max}(|D|) < 1$

Ex: Consider that value of attribute P ranges from -72 to 32 . The maximum absolute value of P is 72. In order to normalize using decimal scaling, the entire range value is individually divided by 100, so that -72 is normalized to -0.72 and 32 normalized to 0.32



For example: suppose that the data for analysis includes the attribute age .

The age values for the data tuples are (increasing order) 13,15,16,16,19,20,20,21,22,22,25,25,25,25,30,33,33,35,35,35,35,36,40,45,46,52,70.

- i. Use min-max normalization to transformation the value 35 for age with range [0.0,1.0]
- ii. Use z-score normalization to transformation the value 35 for age,
where the standard deviation of age is 12.94 years.
- iii. Use normalization by decimal scaling to transform the value 35 for age.

Solutions:

i. $D =$

$$[(35-13)/(70-13)](1.0-0.0)+0.0=0.38$$

$$\text{ii. } D = (35-29.66)/12.94=0.38$$

$$\text{iii. } D = 35/10^Z = 0.35 \text{ here the entire value is divided by } 100$$



Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of **analyzing datasets** to summarize their **main characteristics**, often with **visual methods**. It is typically the **first step** in the data science or machine learning workflow.

EDA helps you understand your data before you apply any machine learning model.

Objectives of EDA

1. Understand the structure and content of the data.
2. Detect outliers, missing values, and anomalies.
3. Identify patterns, trends, and relationships between variables.
4. Generate hypotheses about the data.
5. Guide feature selection and engineering.
6. Ensure data quality before modeling.



Types of EDA

1. Univariate Analysis

Analysis of a **single variable** (feature or column) at a time.

Techniques:

- **Summary statistics:** mean, median, mode, std, min, max
- **Visualizations:**
 - **Histogram:** shows distribution
 - **Boxplot:** detects outliers
 - **Density plot:** smooth curve of distribution

Example: Checking distribution of age in a dataset of customers.

Multivariate Analysis

Analyzing **more than two variables** simultaneously to understand complex interactions.

Techniques:

- **Pair plots** (e.g., sns.pairplot)
- **Heatmaps of correlation matrix**
- **Dimensionality Reduction:**
 - PCA (Principal Component Analysis)
 - t-SNE

Example: Discovering which variables jointly influence customer churn.

Bivariate Analysis

Examines the **relationship between two variables**.

Techniques:

- **Numerical-Numerical:**
 - Scatter plots
 - Correlation coefficients (df.corr())
- **Categorical-Categorical:**
 - Contingency tables (crosstabs)
 - Bar plots
- **Categorical-Numerical:**
 - Box plots
 - Violin plots
 - Group statistics

Example: Analyzing how income varies by education level.



Implement Simple Linear Regression on given Problem

The following table shows the results of a recently conducted study on the correlation of the number of hours spent driving with the risk of developing acute backache. Find the equation of the best fit line for this data.

Number of hours spent driving (x)	Risk score on a scale of 0-100 (y)
10	95
9	80
2	10
15	50
10	45
16	98
11	38
16	93



Theory:

Linear Regression

Regression analysis is used in stats to find trends in data. For example, you might guess that there's a connection between how much you eat and how much you weight; regression analysis can help you quantify that.

In a cause and effect relationship, the **independent variable** is the cause, and the **dependent variable** is the effect. **Least squares linear regression** is a method for predicting the value of a dependent variable Y , based on the value of an independent variable X .

The Least Squares Regression Line:

Linear regression finds the straight line, called the **least squares regression line** or LSRL, that best represents observations in a bivariate data set. Suppose Y is a dependent variable, and X is an independent variable. The population regression line is:

$$Y = B_0 + B_1X$$

Where B_0 is a constant, B_1 is the regression coefficient, X is the value of the independent variable, and Y is the value of the dependent variable.



How to Define a Regression Line:

$$B_1 = \Sigma [(x_i - \bar{x})(y_i - \bar{y})] / \Sigma [(x_i - \bar{x})^2]$$

$$b_1 = r * (s_y / s_x)$$

$$b_0 = \bar{y} - b_1 * \bar{x}$$

where b_0 is the constant in the regression equation,
 b_1 is the regression coefficient,
 r is the correlation between x and y ,
 x_i is the X value of observation i ,
 y_i is the Y value of observation i ,
 $\bar{x}$ is the mean of X , $\bar{y}$ is the mean of Y ,
 s_x is the standard deviation of X ,
and s_y is the standard deviation of Y .

X	Y	(X - $\bar{X}$)	(Y - $\bar{Y}$)	(X - $\bar{X}$)(Y - $\bar{Y}$)	(X - $\bar{X}$) ²	(Y - $\bar{Y}$) ²
10	95	-1.125	31.375	-35.30	1.265	984.39
9	80	-2.125	16.375	-34.80	4.516	268.14
2	10	-9.125	-53.625	489.33	83.265	2875.64
15	50	3.875	-13.625	-52.73	15.015	185.64
10	45	-1.125	-18.625	20.95	1.265	346.89
16	98	4.875	34.875	167.57	23.765	1181.64
11	38	-0.125	-25.625	3.20	0.0156	656.64
16	93	4.875	29.375	143.20	23.765	862.89

$\bar{X} = 11.125$
 $\bar{Y} = 63.625$

$\Sigma = 87.67$ $\Sigma = 19.11$ $\Sigma = 920.23$

$$r = \frac{87.67}{\sqrt{19.11 \times 920.23}} = 0.6611$$

$$S_y = \sqrt{\frac{920.23}{7}} = 11.4656$$

$$S_x = \sqrt{\frac{19.11}{7}} = 1.652$$

$$a = 0.6611 \left(\frac{11.4656}{1.652} \right) = 4.588$$

$$b = 63.625 - (4.58 \times 11.125)$$

$$b = 12.58$$

No. of hours driving x	probability of backache y
10	95
9	80
2	10
15	50
10	45
16	98
11	38
16	93

$$y = ax + b$$

$$b = \bar{y} - a \cdot \bar{x}$$

$$a = r \cdot \frac{S_y}{S_x}$$

$$r = \frac{\Sigma((X - \bar{X})(Y - \bar{Y}))}{\sqrt{\Sigma(X - \bar{X})^2 \cdot \Sigma(Y - \bar{Y})^2}}$$

$$S_y = \sqrt{\frac{\Sigma(Y - \bar{Y})^2}{n-1}} \quad S_x = \sqrt{\frac{\Sigma(X - \bar{X})^2}{n-1}}$$



How to Use the Regression Equation:

$$\hat{y} = b_0 + b_1x$$

$$\hat{y} = 4.588 + 12.58x$$

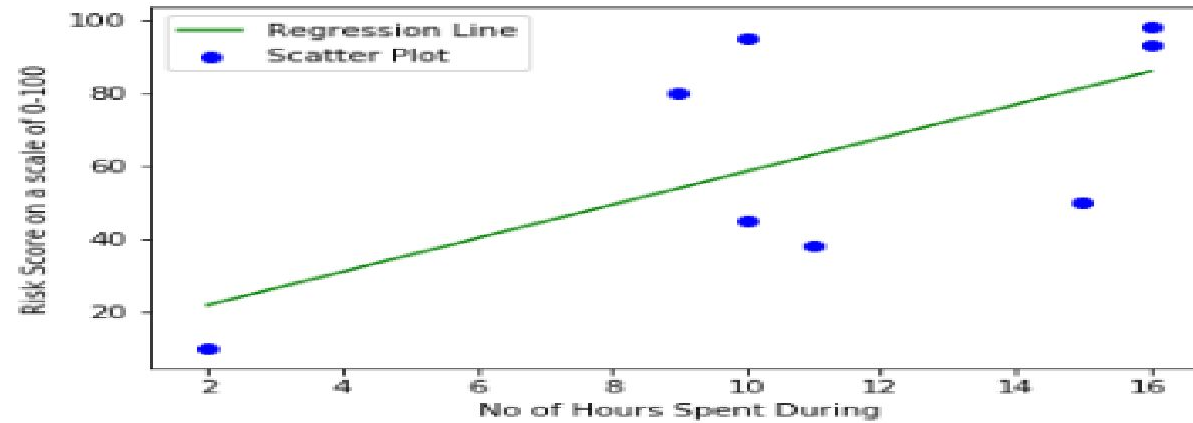
Algorithm:

1. Import the Required Packages
2. Read Given Dataset
3. Import the Linear Regression and Create object of it
4. Find the Accuracy of Model using Score Function
5. Predict the value using Regressor Object
6. Take input from User
7. Calculate the value of y
8. Draw Scatter Plot

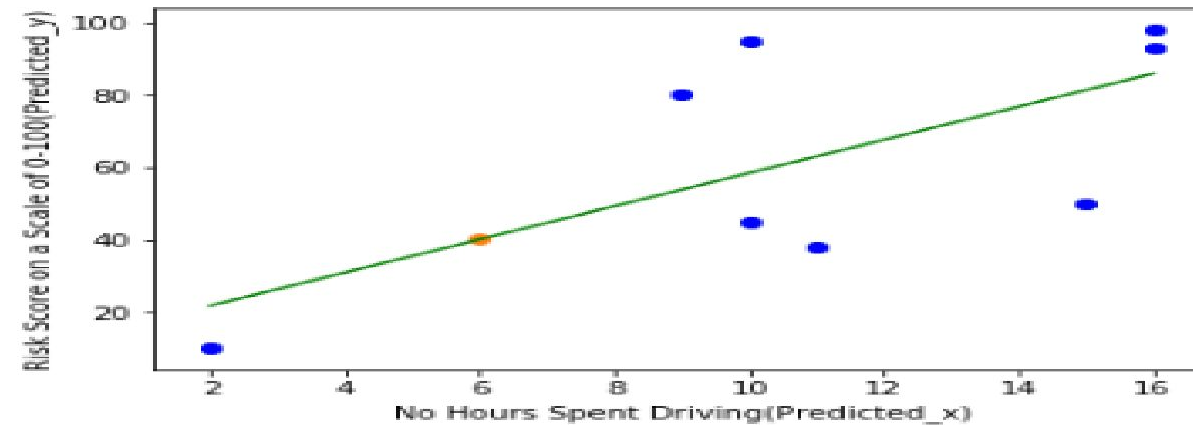


Expected Output:

```
(8, 2)
No of Hours Spent During(X)  Risk Score on a scale of 0-100(Y)
0                             10                             95
1                             9                             80
2                             2                             10
3                             15                             50
4                             10                             45
Slope,Intercept: 4.58789860997547 12.584627964022893
```



```
Root Mean Squares Error: 22.759716640449565
Accuracy: 43.709481451010035
Enter No Hours Spent in Driving:6
```





1. **Trend lines:** A trend line represents the variation in some quantitative data with passage of time (like GDP, oil prices, etc.). These trends usually follow a linear relationship. Hence, linear regression can be applied to predict future values. However, this method suffers from a lack of scientific validity in cases where other potential changes can affect the data.
2. **Economics:** Linear regression is the predominant empirical tool in economics. For example, it is used to predict consumption spending, fixed investment spending, inventory investment, purchases of a country's exports, spending on imports, the demand to hold liquid assets, labor demand, and labor supply.
3. **Finance:** Capital price asset model uses linear regression to analyze and quantify the systematic risks of an investment.
4. **Biology:** Linear regression is used to model causal relationships between parameters in biological systems.



Linear Regression for House Price Prediction:

Build and evaluate a linear regression model to predict house prices based on size, location, number of rooms, and amenities. Evaluate the model.

```
[ ] # Step 2: Create Dataset (Synthetic Example)
data = {
    'size': [1500, 1800, 2400, 3000, 3500, 4000, 1200, 1400, 2000, 2200],
    'location': [1, 2, 2, 3, 3, 1, 1, 2, 3, 1], # Categorical encoded as integers
    'rooms': [3, 4, 3, 5, 4, 3, 2, 2, 4, 3],
    'amenities': [7, 8, 6, 9, 10, 7, 5, 6, 8, 7],
    'price': [300000, 350000, 450000, 500000, 600000, 420000, 250000, 280000, 480000, 390000]
}

df = pd.DataFrame(data)
display(df.head())
```



	size	location	rooms	amenities	price
0	1500	1	3	7	300000
1	1800	2	4	8	350000
2	2400	2	3	6	450000
3	3000	3	5	9	500000
4	3500	3	4	10	600000


```
# Step 4: Split Data into Training and Testing Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("X_train values:\n", X_train)
print("X_test values:\n", X_test)
```

```
⇒ X_train values:
   size  location  rooms  amenities
5  4000           1      3          7
0  1500           1      3          7
7  1400           2      2          6
2  2400           2      3          6
9  2200           1      3          7
4  3500           3      4         10
3  3000           3      5          9
6  1200           1      2          5
X_test values:
   size  location  rooms  amenities
8  2000           3      4          8
1  1800           2      4          8
```



```
[ ] # Step 6: Make Predictions
y_pred = model.predict(X_test)

print("Predicted Prices:", y_pred)
```

⇒ Predicted Prices: [453180.08575504 397493.53712528]

```
▶ # Step 7: Evaluate Model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Display Results
print("Actual Prices:", y_test.values)
print("Predicted Prices:", np.round(y_pred).astype(int))
print("Mean Squared Error (MSE):", mse)
print("R-squared (R2) Score:", r2)
```

⇒ Actual Prices: [480000 350000]
Predicted Prices: [453180 397494]
Mean Squared Error (MSE): 1487471934.3888688
R-squared (R2) Score: 0.6479356368310369

```
[ ] # Step 8 (Optional): Present in Table Form
results_df = pd.DataFrame({
    'Actual Price': y_test.values,
    'Predicted Price': np.round(y_pred).astype(int)
})
display(results_df)
```

⇒

	Actual Price	Predicted Price
0	480000	453180
1	350000	397494



Logistic Regression

Logistic regression is the appropriate regression analysis to conduct when the dependent variable is dichotomous (binary).

Like all regression analyses, the logistic regression is a predictive analysis.

Logistic regression is used to describe data and to explain the relationship between one dependent binary variable and one or more nominal, ordinal, interval or ratio-level independent variables.

Remember: though the name of algorithm carries regression, it is used for **classification**.



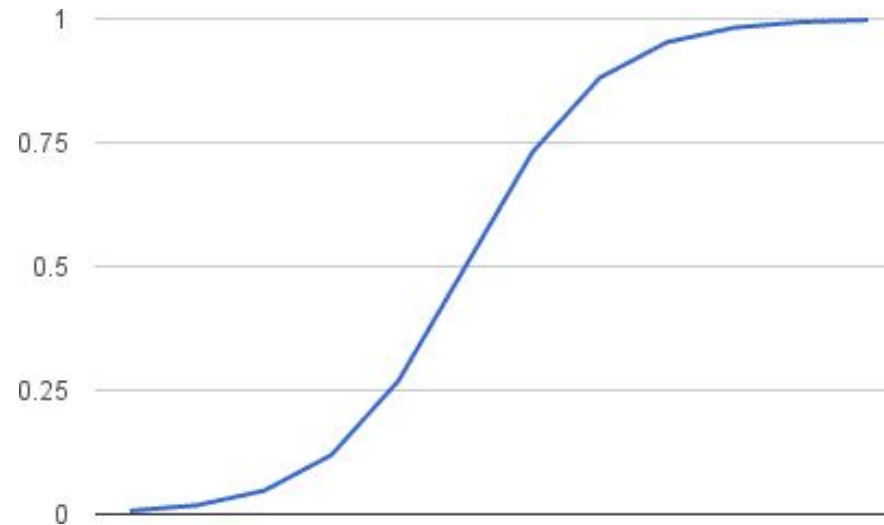
Logistic regression is named for the function used at the core of the method, the logistic function.

The logistic function, also called the **sigmoid function** was developed by statisticians to describe properties of population growth in ecology, rising quickly and maxing out at the carrying capacity of the environment.

It's **an S-shaped curve** that can take any real-valued number and map it into a value between 0 and 1, but never exactly at those limits.

$$1 / (1 + e^{-\text{value}})$$

e is the base of the natural logarithms (Euler's number or the EXP() function in your spreadsheet) and value is the actual numerical value that you want to transform. Below is a plot of the numbers between -5 and 5 transformed into the range 0 and 1 using the logistic function.





Input values (x) are combined linearly using weights or coefficient values (referred to as the Greek capital letter Beta) to predict an output value (y).

A key difference from linear regression is that the output value being modeled is a binary values (0 or 1) rather than a numeric value.

Below is an example logistic regression equation:

$$y = e^{(b_0 + b_1 * x)} / (1 + e^{(b_0 + b_1 * x)})$$



Logistic regression models the probability of the default class (e.g. the first class).

For example, if we are modeling people's gender as male or female from their height, then the first class could be male and the logistic regression model could be written as the probability of male given a person's height, or more formally:

$$P(\text{gender}=\text{male}|\text{height})$$

Written another way, we are modeling the probability that an input (X) belongs to the default class (Y=1), we can write this formally as:

$$P(X) = P(Y=1|X)$$



Let's say we have a model that can predict whether a person is male or female based on their height (completely fictitious). Given a height of 150cm is the person male or female.

We have learned the coefficients of $b_0 = -100$ and $b_1 = 0.6$. Using the equation above we can calculate the probability of male given a height of 150cm or more formally $P(\text{male} | \text{height}=150)$. We will use $\text{EXP}()$ for e , because that is what you can use if you type this example into your spreadsheet:

$$y = e^{(b_0 + b_1 * X)} / (1 + e^{(b_0 + b_1 * X)})$$

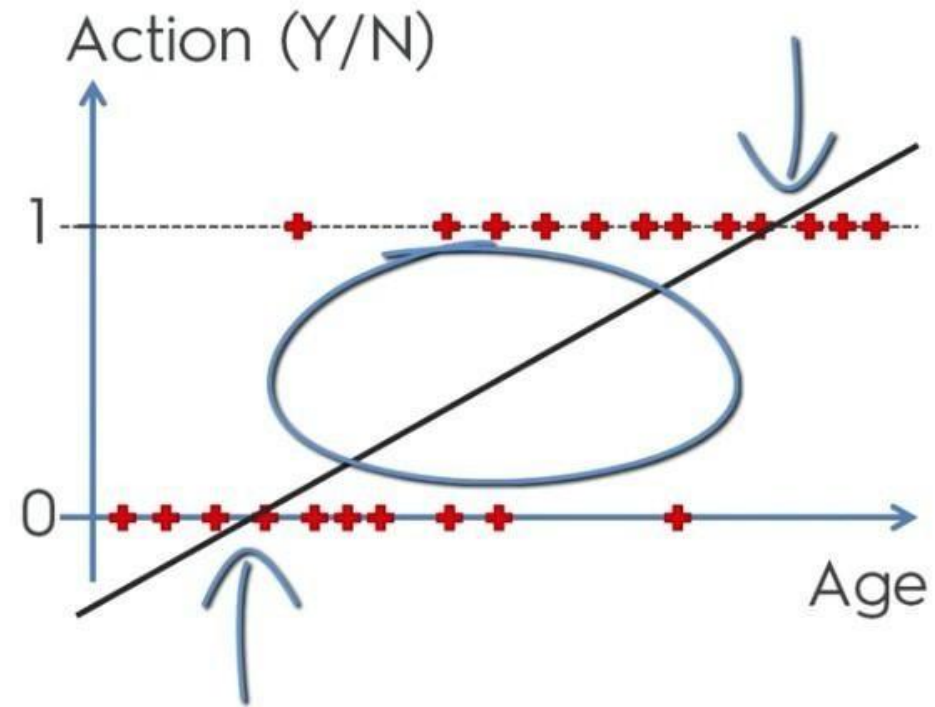
$$y = \exp(-100 + 0.6 * 150) / (1 + \text{EXP}(-100 + 0.6 * X)) \quad y = 0.0000453978687$$

Or a probability of near zero that the person is a male.

In practice we can use the probabilities directly. Because this is classification and we want a crisp answer, we can snap the probabilities to a binary class value, for example:

1 if $p(\text{male}) < 0.5$
2 if $p(\text{male}) \geq 0.5$

For ex: A company has provided an offer by email to their customers.



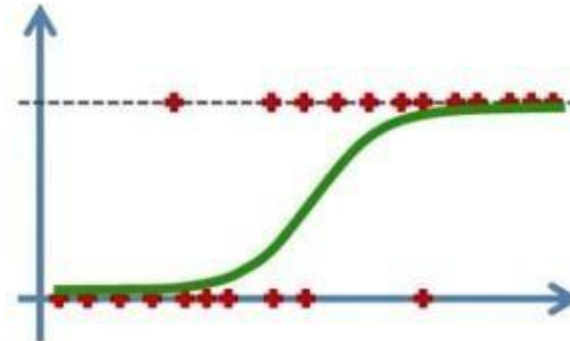
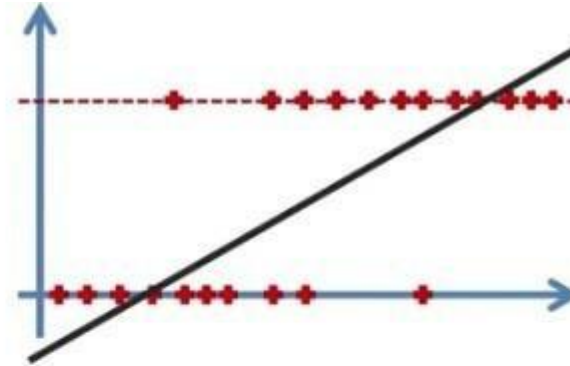
Logistic Regression

$$y = b_0 + b_1 * x$$

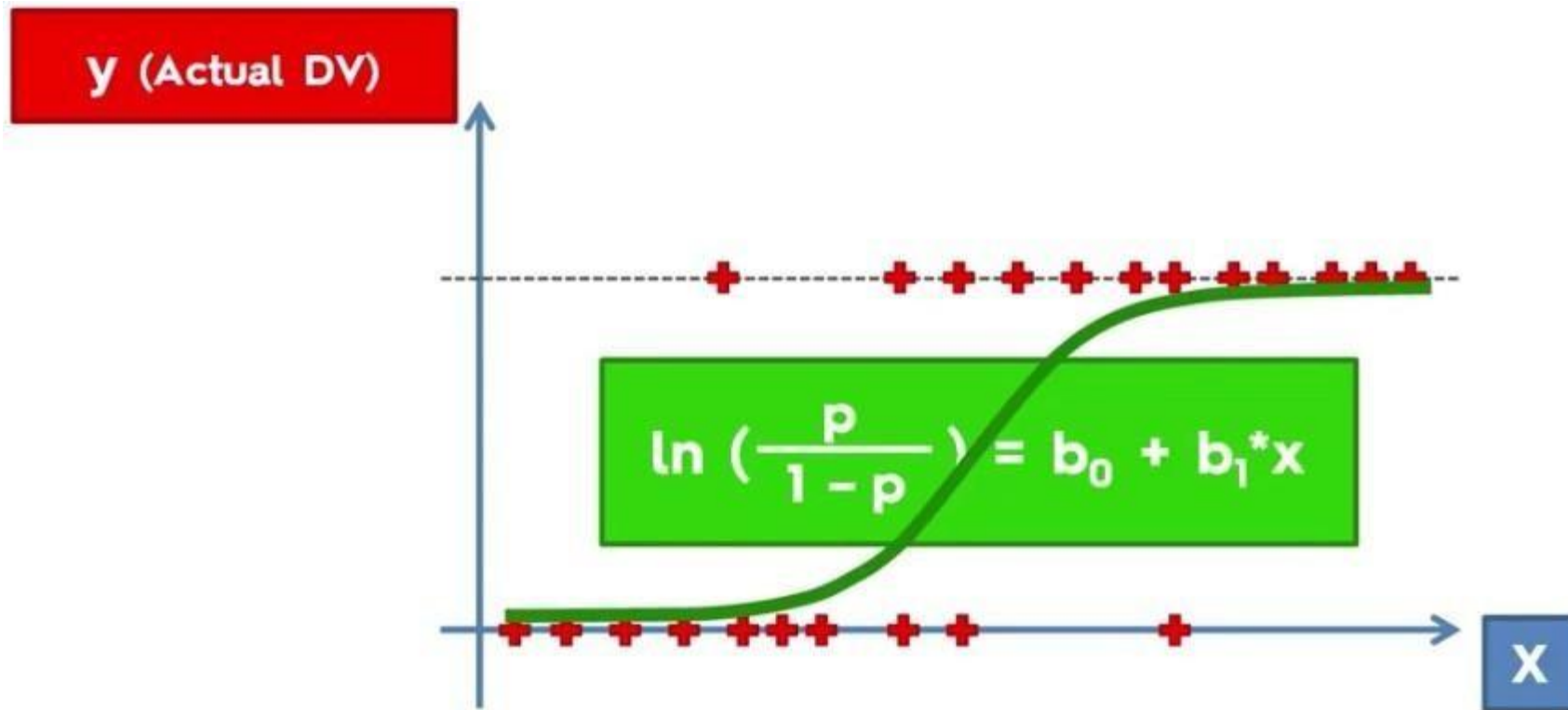
Sigmoid Function

$$p = \frac{1}{1 + e^{-y}}$$

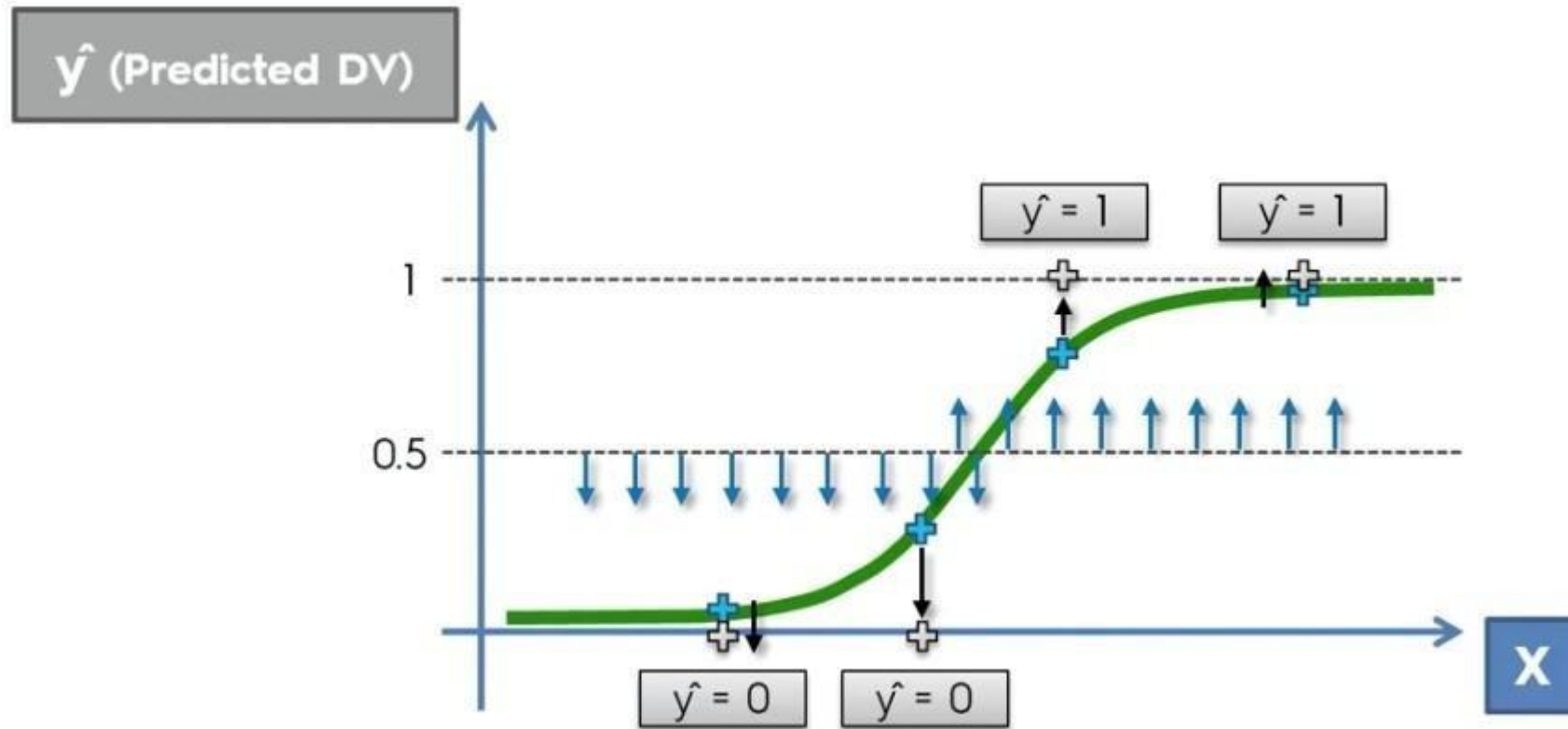
$$\ln \left(\frac{p}{1 - p} \right) = b_0 + b_1 * x$$



Logistic Regression – Logit Function



Logistic Regression – Prediction





Advantages

- Logistic Regression performs well when the dataset is **linearly separable**.
- Logistic regression is **less prone to over-fitting** but it can overfit in high dimensional datasets. You should consider Regularization (L1 and L2) techniques to avoid over-fitting in these scenarios.
- Logistic Regression not only gives a measure of how relevant a predictor (coefficient size) is, but also its **direction of association** (positive or negative).
- Logistic regression is **easier** to implement, interpret and very **efficient** to train.



Disadvantages

- Main limitation of Logistic Regression is the **assumption of linearity** between the dependent variable and the independent variables. In the real world, the data is rarely linearly separable. Most of the time data would be a jumbled mess.
- If the number of observations are lesser than the number of features, Logistic Regression should not be used, otherwise it may lead to **overfit**.
- Logistic Regression can only be used to **predict discrete functions**. Therefore, the dependent variable of Logistic Regression is restricted to the discrete number set.



Aim: Implement Decision Tree on given Problem

A dataset collected in a cosmetics shop showing details of customers and whether or not they responded to a special offer to buy a new lip-stick is shown in table below. Use this dataset to **build a decision tree**, with Buys as the target variable, to help in buying lip-sticks in the future. **Find the root node of decision tree**. According to the decision tree you have made from previous training data set, what is the decision for the test data:
[Age < 21, Income = Low, Gender = Female, Marital Status = Married]?

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Decision Tree is a decision-making tool that uses a flowchart-like tree structure or is a model of decisions and all of their possible results, including outcomes, input costs and utility. Decision-tree algorithm falls under the category of supervised learning algorithms. It works for both continuous as well as categorical output variables.

The branches/edges represent the result of the node and the nodes have either:

1. Conditions [Decision Nodes]
2. Result [End Nodes]

A decision tree can be visualized. A decision tree is one of the many Machine Learning algorithms.



Step1:Compute the Entropy for the data set

Out of 14 instances we have 9 YES and 5 NO

So we have the formula,

$$E(S) = -P(\text{Yes}) \log_2 P(\text{Yes}) - P(\text{No}) \log_2 P(\text{No})$$

$$E(S) = - (9/14)^* \log_2 9/14 - (5/14)^* \log_2 5/14$$

$$E(S) = 0.41 + 0.53 = 0.94$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?

Age

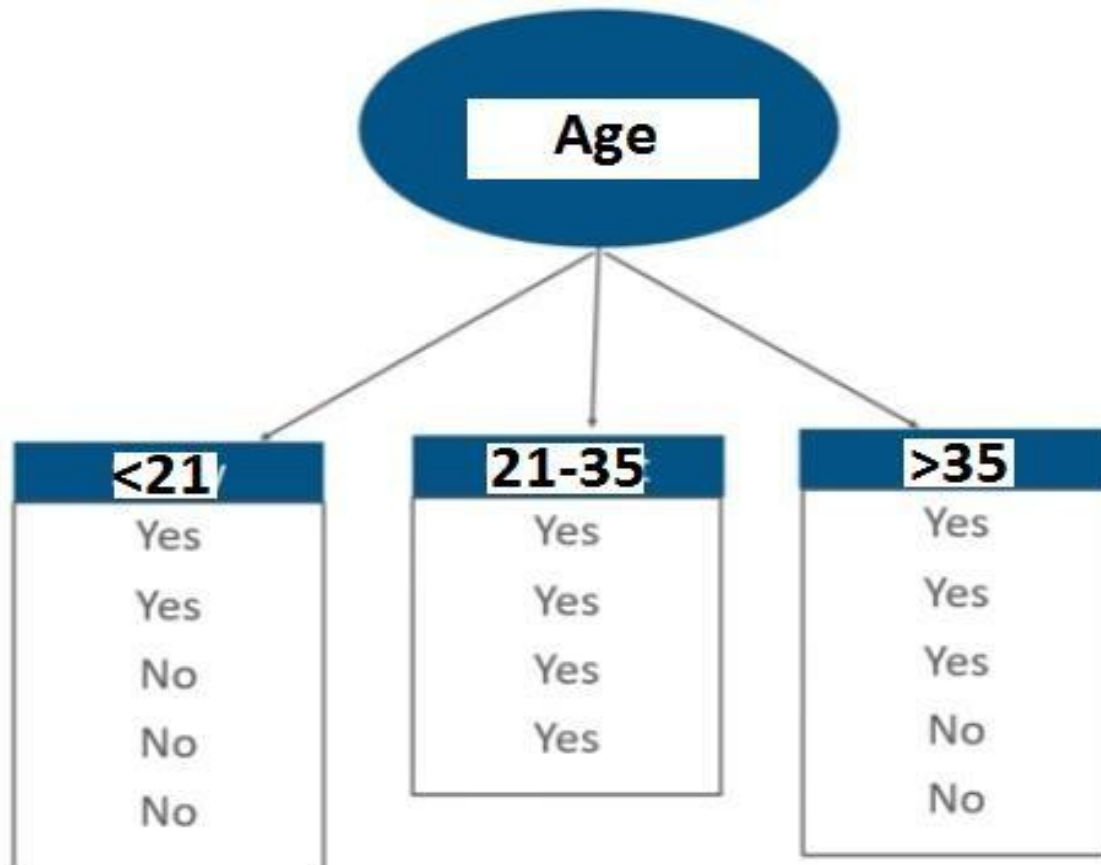
Income

gender

Marital
Status

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

Which node to select as root node?



ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?

$$E(\text{age} = <21) = -2/5 \log_2 2/5 - 3/5 \log_2 3/5 = 0.971$$

$$E(\text{age} = 21-35) = -1 \log_2 1 - 0 \log_2 0 = 0$$

$$E(\text{age} = >35) = -3/5 \log_2 3/5 - 2/5 \log_2 2/5 = 0.971$$

Information from outlook,

$$I(\text{age}) = 5/14 \times 0.971 + 4/14 \times 0 + 5/14 \times 0.971 = 0.693$$

Information gained from age

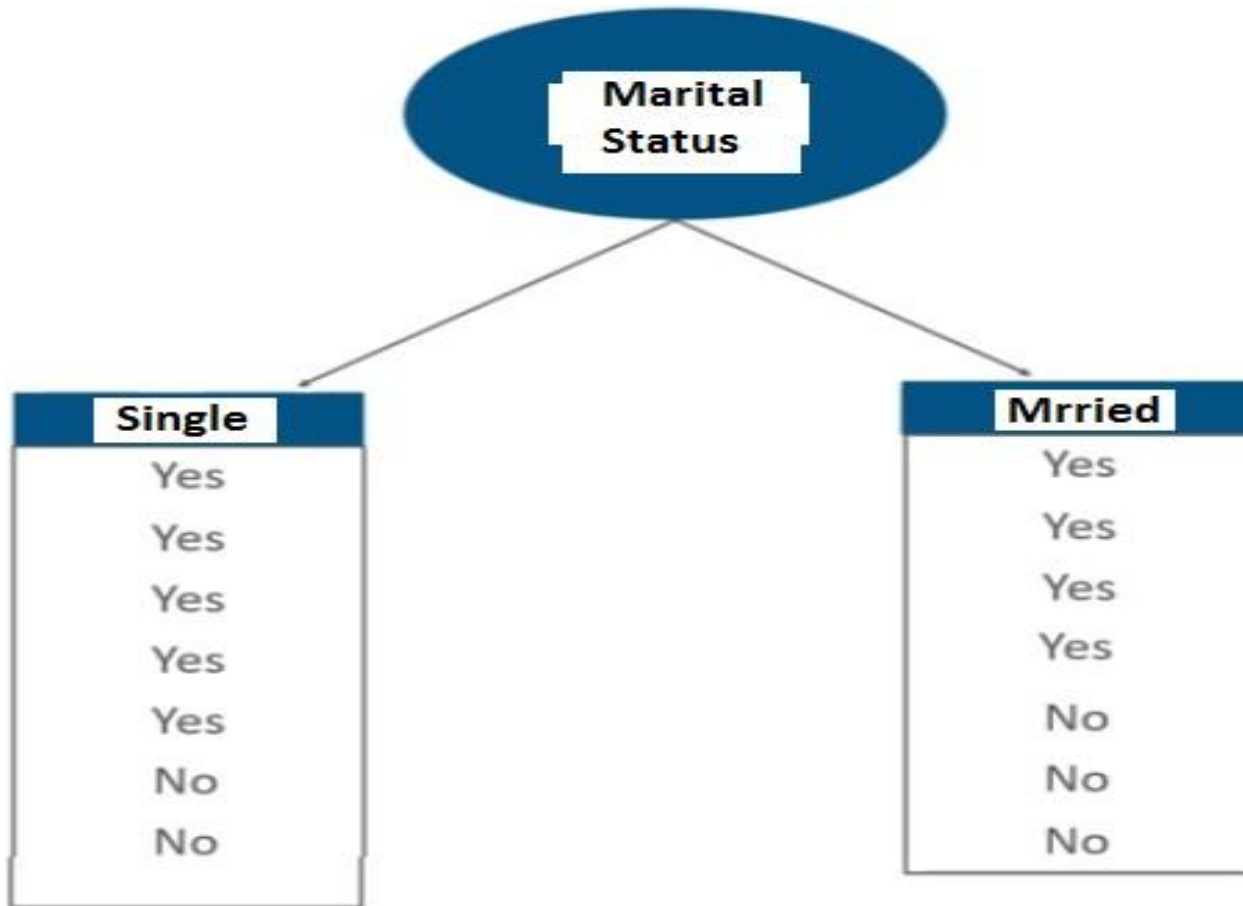
$$\text{Gain}(\text{age}) = E(S) - I(\text{age})$$

$$0.94 - 0.693 = 0.247$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?



ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No



Which node to select as root node?

$$E(\text{Marital} = \text{True}) = 1$$

status

$$E(\text{Marital} = \text{False}) = 0.811$$

status

Information from windy,

$$I(\text{Marital Status}) = 8/14 \times 0.811 + 6/14 \times 1 = 0.892$$

Information gained from outlook,

$$\text{Gain}(\text{Marital Status}) = E(S) - I(\text{Marital Status})$$

$$0.94 - 0.892 = 0.048$$

ID	Age	Income	Gender	Marital Status	Buys
1	< 21	High	Male	Single	No
2	< 21	High	Male	Married	No
3	21-35	High	Male	Single	Yes
4	>35	Medium	Male	Single	Yes
5	>35	Low	Female	Single	Yes
6	>35	Low	Female	Married	No
7	21-35	Low	Female	Married	Yes
8	< 21	Medium	Male	Single	No
9	<21	Low	Female	Married	Yes
10	> 35	Medium	Female	Single	Yes
11	< 21	Medium	Female	Married	Yes
12	21-35	Medium	Male	Married	Yes
13	21-35	High	Female	Single	Yes
14	> 35	Medium	Male	Married	No

With similar Calculations we get

Gain (Age) = 0.247 (root) Gain(Income)= 0.029 Gain(Gender)= 0.024 Gain(Marital Status)=0.048



Explore practical Implementation

Source Code :

Import packages

import numpy as np

import matplotlib.pyplot as

plt import pandas as pd

#reading Dataset

dataset=pd.read_csv("data.csv

") X=dataset.iloc[:, :-1]

y=dataset.iloc[:,5].values

#Perform Label encoding

from sklearn.preprocessing import

LabelEncoder labelencoder_X = LabelEncoder()

X = X.apply(LabelEncoder().fit_transform)

print (X)

from sklearn.tree import

DecisionTreeClassifier

regressor=DecisionTreeClassifier()

regressor.fit(X.iloc[:,1:5],y)

#Predict value for the given expression

X_in=np.array([0,0,0,1])

y_pred=regressor.predict([X_in

]) print ("Prediction:", y_pred)

from sklearn.externals.six import StringIO

from IPython.display import Image

from sklearn.tree import export_graphviz

import pydotplus

Create DOT data

dot_data = StringIO()

export_graphviz(regressor, out_file=dot_data, filled=True,
rounded=True, special_characters=True)

Draw graph

graph = pydotplus.graph_from_dot_data(dot_data.getvalue())

graph.write_png('Decision_Tree.png')

Show graph

Image(graph.create_png())



Expected Output

	id	age	income	gender	marital_status
0	0	1	0	1	1
1	1	1	0	1	0
2	2	0	0	1	1
3	3	2	2	1	1
4	4	2	1	0	1
5	5	2	1	0	0
6	6	0	1	0	0
7	7	1	2	1	1
8	8	1	1	0	0
9	9	2	2	0	1
10	10	1	2	0	0
11	11	0	2	1	0
12	12	0	0	0	1
13	13	2	2	1	0

Prediction: ['Yes']



Decision Trees

Advantages

- Decision trees are powerful and popular tools for classification and prediction.
- Simpler and ease of use.
- They are able to handle both numerical and categorical attributes.
- Easy to understand.
- State is recorded in memory.
- Provide a clear indication of which fields are most important for prediction or classification



DECISION TREE APPLICATIONS

- Medical diagnosis.
- Credit risk analysis.
- Library book use Etc.



Support Vector Machine Algorithm:

Support Vector Machine Algorithm:

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane



SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

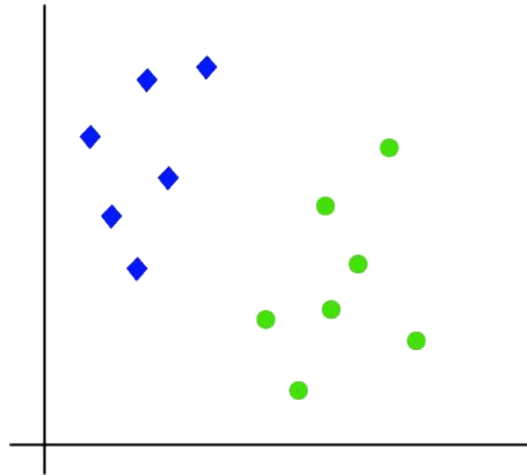
Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

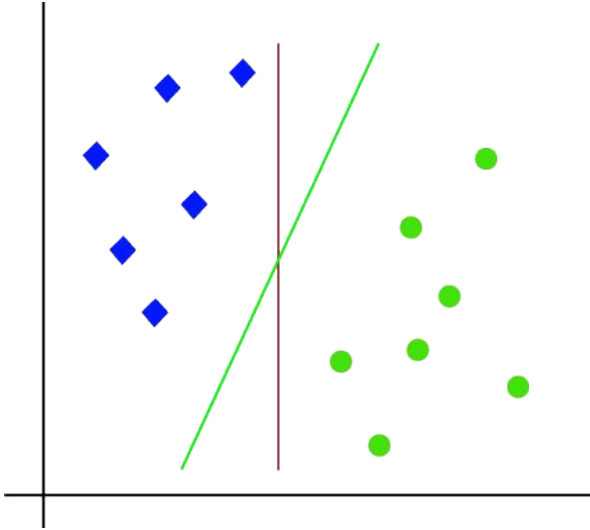
We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Linear SVM:

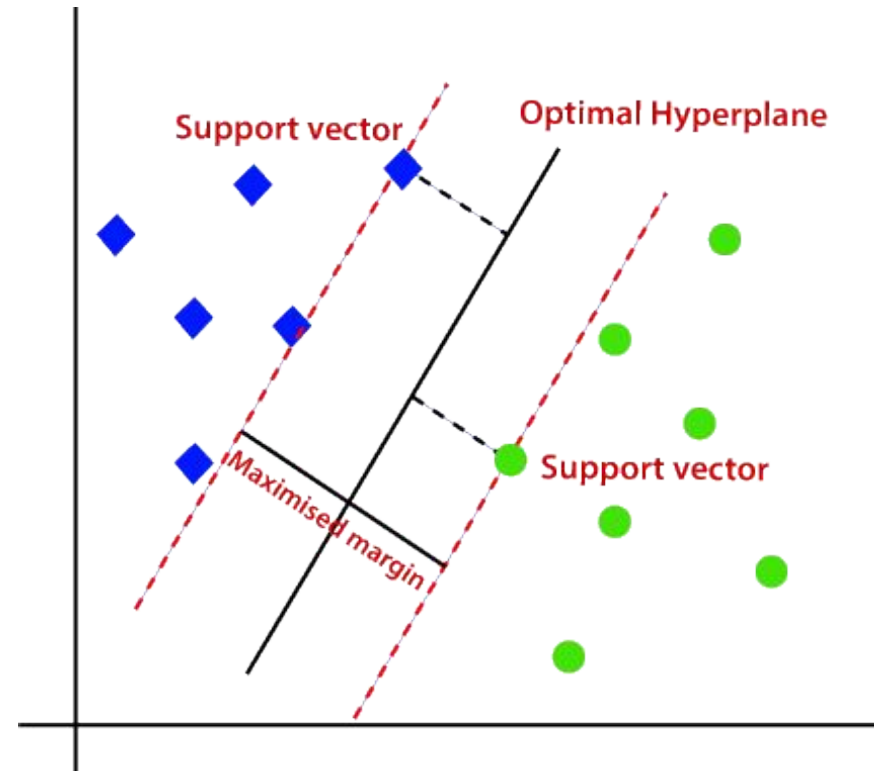
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:



Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Algorithm Steps:

- • Load the breast cancer dataset from sklearn.datasets
- • Separate input features and target variables.
- • Build and train the SVM classifiers using RBF kernel.
- • Plot the scatter plot of the input features.
- • Plot the decision boundary.
- • Plot the decision boundary



Model Evaluation and Validation Techniques

Overfitting and Underfitting:

- Overfitting: Model learns noise and details in training data, leading to poor generalization on unseen data.
- Underfitting: Model too simple to capture underlying data patterns, resulting in poor performance on both training and test data.

Cross-Validation:

- Technique to evaluate model generalization by partitioning data into training and validation sets multiple times.
- k-Fold Cross-Validation:
 - Split data into k equal parts (folds).
 - Train on $k-1$ folds and validate on the remaining fold.
 - Repeat k times with each fold as validation once.
 - Average the performance metrics.
- Helps detect overfitting and model stability.



Common Model Evaluation Metrics:

For classification problems:

- **Accuracy:** Overall correctness
- **Precision:** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
- **Recall (Sensitivity):** $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
- **F1 Score:** Harmonic mean of precision and recall
- **Confusion Matrix:** Counts of TP, TN, FP, FN
- **ROC Curve & AUC:** Evaluates tradeoff between true positive rate and false positive rate across thresholds



Real-World Example: Medical Test for COVID-19

Suppose we are testing 100 people for COVID-19. Here's what actually happens:

- 30 people **actually have COVID-19**
- 70 people **do not have COVID-19**

Now suppose the test gives these results:

25 of the 30 sick people test positive →  **True Positives (TP) = 25** (Correctly caught the sick person)

5 of the sick people test negative →  **False Negatives (FN) = 5** (Mistakenly said a healthy person is sick)

60 of the healthy people test negative →  **True Negatives (TN) = 60** (Correctly said a healthy person is healthy)

10 of the healthy people test positive →  **False Positives (FP) = 10** (Missed a sick person)



Confusion Matrix

	Actually Has COVID	Actually Doesn't Have COVID
Test Says Positive	✓ TP = 25	✗ FP = 10
Test Says Negative	✗ FN = 5	✓ TN = 60

Now Let's Calculate Some Metrics:

$$\text{Accuracy} = (TP + TN) / \text{Total} = (25 + 60) / 100 = 85\%$$

$$\text{Precision} = TP / (TP + FP) = 25 / (25 + 10) = 71.4\%$$

$$\text{Recall (Sensitivity)} = TP / (TP + FN) = 25 / (25 + 5) = 83.3\%$$

$$\begin{aligned} \text{F1 Score} &= 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 2 \times \\ &(0.714 \times 0.833) / (0.714 + 0.833) \approx 0.768 \text{ or } 76.8\% \end{aligned}$$



Regularization and Validation in Machine Learning

Why It Matters:

In machine learning, **overfitting** happens when a model learns the noise in the training data rather than the underlying pattern. **Underfitting**, on the other hand, happens when the model is too simple to capture the data's complexity.

To build **robust models** that generalize well to unseen data, we use:

Regularization Techniques

Regularization adds a penalty to the model's loss function to **discourage complexity**:

➤ L1 Regularization (Lasso):

- Adds the **absolute value** of coefficients as a penalty term.
- Promotes **sparse models** (some coefficients become exactly zero).
- Used when **feature selection** is important.

➤ L2 Regularization (Ridge):

- Adds the **squared value** of coefficients as a penalty.
- Shrinks coefficients smoothly but does **not eliminate** them entirely.
- Useful when you want to **reduce model complexity** without dropping features.

➤ Elastic Net:

- Combines L1 and L2 penalties.
- Useful when you have **many correlated features**.



Validation Techniques

Validation helps **tune hyperparameters** and assess how well your model generalizes:

➤ **Holdout Validation:**

- Split data into training and validation sets (e.g., 80/20).
- Simple but may not represent all data patterns.

➤ **K-Fold Cross-Validation:**

- Split data into k folds, train on $k-1$, validate on 1, repeat.
- Reduces bias due to random train/validation splits.
- Provides a more **stable estimate of performance**.

➤ **Grid Search / Random Search with Cross-Validation:**

- Tune hyper parameters like regularization strength (e.g., C in Logistic Regression or SVM).
- Evaluate each combination using cross-validation.



Applied to Common Models

Logistic Regression

- Use **L1 or L2 regularization** to prevent overfitting.
- Tune **C** (inverse of regularization strength).

Decision Trees

- Prone to overfitting.
- Use parameters like *max_depth*, *min_samples_split*, and **pruning**.
- Validation is critical to avoid overly complex trees.

Support Vector Machines (SVMs)

- Use **C** for regularization (lower C = stronger regularization).
- Use **kernel trick** with validation to choose the right kernel (linear, RBF, etc.).



Suggested Readings

Text Books:

1. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron - Covers Unit I & II: Foundational ML concepts, supervised & unsupervised learning, and data preprocessing.
2. Pattern Recognition and Machine Learning" by Christopher M. Bishop - Supports Unit I & IV: Deep dive into mathematical foundations and evaluation techniques.
3. Machine Learning Engineering" by Andriy Burkov - Excellent for Unit III: Practical model deployment, CI/CD, and ML systems in industry
4. Deep Learning with Python" by François Chollet - Addresses Unit V: Includes foundation models, generative AI, and on-device deep learning.

Reference Books:

1. Introduction to Statistical Learning" by Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani - Relevant for Units I & II: Statistical methods, linear models, and feature engineering.
2. Designing Machine Learning Systems with Python" by David Julian - Supports Unit III: Implementation, pipelines, and deployment using industry tools
3. Ethics of Artificial Intelligence and Robotics" (Stanford Encyclopedia of Philosophy) - Unit IV: Ethical, societal, and legal implications of ML and AI.
4. Graph Representation Learning" by William L. Hamilton - Unit V: In-depth reference on Graph Neural Networks and their applications.